

Difficulty Report For The Current PDF-to-SHACL System

Mid Term 1 Accessibility Checking Prototype

12 June 2026

1 What This Report Is About

This report explains the current system in plain words. It also explains which parts were hard, what was tried, what works now, and what is still weak.

2 Agents In The Current System

The current system has **5 agent-like decision components**. Only two of them are literally named “Agent” in the Python code, but all five make decisions or produce decision-ready output.

Agent or decision component	Input	Output	Main parameters it uses
Vision Observation Agent	Selected PDF image chunks. Each image has page number, image index, image path, OCR text, and image ranking meta-data.	Vision text. This is a written description of visible measurements, symbols, tables, diagrams, labels, limits, ranges, and rule wording.	Image rank, image path, page number, model name qwen3-v1:8b , maximum selected images, prompt text, timeout, and whether the image was selected for vision.
Requirement Extraction Agent	Selected evidence chunks from text, tables, OCR, and vision.	Canonical rule candidates. These are neutral rules with rule ID, entity, property, operator, value, unit, source text, source page, confidence, and evidence chunk ID.	Evidence rank, chunk type, page number, evidence text, batch size, canonical rule schema, JSON output shape, model name qwen3.5:9b , temperature 0.0, and rejection rules for invalid output.
Candidate Governance Selector	All canonical rule candidates produced by the extraction agent.	Best selected canonical rule per rule type, duplicate groups, conflict records, rejected competitors, and selection explanation.	Semantic relevance, measurement specificity, source specificity, requirement completeness, ontology consistency, retrieval support, evidence density, contradiction penalty, confidence, source page, and evidence chunk ID.

Ontology Mapping Agent	Selected canonical rules and the enriched RDF graph from the IFC model.	Mappings from canonical rules to RDF classes and RDF properties. For example, a canonical width rule must point to the RDF property that stores that width.	Rule ID, entity, property, candidate ontology class, candidate ontology property, mapping score, graph availability, mapping review status, and mapping audit data.
Human Review And Approval Workflow	Selected rules, duplicate groups, conflict groups, and ontology mappings.	Approved, rejected, or escalated review records. Only approved rules with approved mappings can be published into SHACL.	Reviewer name, timestamp, rationale, candidate ID, rule ID, evidence chunk, source page, confidence, retrieval score, selection score, duplicate status, conflict status, and mapping decision.

In short: the vision component reads selected images, the extraction agent turns evidence into rule candidates, the governance selector chooses the best-supported candidates, the ontology mapping agent connects those rules to RDF properties, and the human review workflow decides what is allowed to become SHACL.

Some technical words are used because the project needs them:

- **IFC**: a building model file. It stores building objects such as doors, rooms, floors, stairs, ramps, and walls.
- **RDF**: a graph format for data. It stores facts as triples, for example: a door has a width.
- **SHACL**: a rule language for RDF graphs. It says what data must satisfy, for example: door width must be at least 0.90 m.
- **pySHACL**: the Python tool that runs SHACL rules on RDF data and produces a report.
- **Canonical rule**: the neutral rule format used before SHACL. It is not tied to RDF, SHACL, or one ontology.
- **Ontology mapping**: the step that connects a canonical rule such as `door_width` to the RDF property that stores door width in the graph.
- **OCR**: text recognition from images.
- **Vision model**: a model that looks at an image and writes useful observations about it.

The report is based on the current files and latest run outputs:

- `output/final_system_correctness_audit.json`
- `output/app_package/rule_extraction_audit.json`
- `output/app_package/document_chunks.json`
- `output/app_package/human_review_audit.json`
- `output/app_package/generated_accessibility_rules.shacl.ttl`
- `output/app_package/shacl_report.ttl`

3 Latest Real Run

The latest full run used:

- PDF: `planungsgrundlagen_barrierefreies_bauen.pdf`
- IFC: `AC20-Institute-Var-2.ifc`
- Text model: `qwen3.5:9b`
- Vision model: `qwen3-vl:8b`
- OCR: `paddleocr` with `paddlepaddle==3.2.2`

The measured result was:

- 65 PDF pages were read.
- 1111 text chunks were created.

- 53 table chunks were created.
- 210 image evidence chunks were created.
- 120 chunks came from embedded PDF images.
- 65 chunks came from rendered full PDF pages.
- 25 chunks came from rendered table regions.
- 210 image chunks were ranked.
- 30 images were sent to the vision model.
- 191 images had OCR text.
- 28 images had vision text.
- 48 calls were made to the text model.
- 14 model outputs were accepted as rule candidates.
- 7 model outputs were rejected.
- 4 final canonical rules were selected.
- 317 route edges were created from the IFC file.
- 0 SHACL issues were reported.
- 0 missing geometry records were reported.

4 How The System Works

The PDF side works like this:

PDF

- > split into pages, text, tables, and images
- > run OCR on image text
- > run vision model on selected images
- > rank useful evidence
- > send evidence to the rule extraction agent
- > create canonical rule candidates
- > select the best-supported candidate
- > review and approve rules
- > map rules to RDF properties
- > generate SHACL
- > run pySHACL validation

The IFC side works like this:

IFC model

- > RDF graph from IFCtoLBD
- > add geometry and route measurements
- > run SHACL rules on the RDF graph
- > write app data for the browser

5 Parameters Used By Each Component

This table lists what each component actually looks at. These are not magic values. They are the data fields, scores, and checks used by the system.

Component	What it looks at
PDF ingestion	PDF file path, page number, page width, page height, page rotation, text length, image count, table count, PDF metadata, and output folder.

Text chunks	Page number, text block content, cleaned whitespace, chunk ID, optional bounding box, text length, and page grouping. A chunk is a smaller piece of the page.
Table chunks	Page number, table text, row count, column count, extraction method, optional bounding box, measurement-heavy text, and surrounding page text.
Image extraction	Page number, image index, image width, image height, file type, image path, optional bounding box, OCR text, vision text, provider status, and processing errors.
Image ranking	Measurement density, numeric density, normative language, OCR text quality, page context density, table context, visual salience, uniqueness, final score, rank, selection reason, and exclusion reason.
OCR	Image path, page number, image index, OCR provider name, OCR result text, execution status, empty results, and errors.
Vision model	Image path, page number, image index, selected-for-vision flag, model name, maximum selected images, prompt text, returned observation text, timeout, and errors.
Evidence retrieval	Measurement density, constraint likelihood, normative language, definition likelihood, table importance, cross-reference score, citation score, requirement confidence, evidence uniqueness, semantic centrality, rank, selected/excluded status, and reason.
Prompt assembly	Selected chunk IDs, chunk type, page number, evidence text, batch index, batch size, canonical rule list, JSON output shape, and instruction that the model must not generate SHACL or RDF.
Text model provider	Model name qwen3.5:9b , Ollama host, JSON mode, thinking disabled, temperature 0.0, timeout, raw response, JSON parse result, and provider name.
Requirement extraction agent	Evidence chunk ID, evidence type, evidence text, source page, raw model output, rule ID, entity, property, operator, value, unit, source text, confidence, candidate ID, accepted/rejected status, and rejection reason.
Canonical rule normalization	Configured rule ID, description, entity, property, operator, expected value, expected unit, tolerance, unit conversion, and supported/unsupported decision. These come from rules/canonical_rule_schema.json .
Candidate governance	Candidate ID, rule ID, confidence, evidence chunk ID, source page, semantic relevance, measurement specificity, source specificity, requirement completeness, ontology consistency, retrieval support, evidence density, contradiction penalty, final selection score, and explanation.
Duplicate handling	Rule ID, candidate values, units, operators, duplicate group ID, group members, selected winner, rejected candidates, and merge reason.
Conflict handling	Rule ID, different values for the same rule, operators, supporting evidence, confidence comparison, conflict ID, conflict type, group members, and review status.
Human review	Review ID, review type, candidate ID, rule ID, mapping ID, evidence chunk, source page, confidence, retrieval score, selection score, reviewer, timestamp, rationale, decision, and evidence references.
Rule repository	Canonical rules, source PDF path, extraction metadata, chunk counts, document metadata, audit paths, source text, source page, confidence, candidate ID, and evidence references.

Ontology mapping	Rule ID, entity, property, target ontology class, target RDF property, mapping candidate ID, mapping score, mapping status, mapping review decision, and audit record.
SHACL generation	Approved rule, approved mapping, rule ID, candidate ID, evidence chunk ID, source page, review metadata, target class, target property, operator, numeric value, unit, datatype, NodeShape, PropertyShape, provenance triples, and Turtle output path.
Publication gate	Approved review exists, review queue exists, decision records exist, no unresolved conflicts, no unresolved duplicate groups, evidence exists, source chunk exists, confidence threshold, provenance complete, mapping approved, and generated SHACL valid.
pySHACL validation	Data graph path, SHACL shapes path, report path, graph parsing result, target nodes, constraint results, conforms flag, and issue conversion for the app.
IFC geometry	IFC path, element type, GlobalId, name, floor, bounding box, center point, dimensions, declared width, derived door width, clear space width, turning space, ramp length, ramp width, ramp slope, and missing geometry records.
IFCtoLBD RDF workflow	IFC path, IFCtoLBD zip path, temporary folder, raw RDF output, Java process result, RDF load result, ontology warnings, raw graph path, enriched graph path, and graph format.
Route graph	IfcRelSpaceBoundary links, door-to-space mapping, spaces per door, doors per space, floor grouping, route edge ID, start door, end door, path, width, door width, turning space, ramp intersection, stair intersection, route status, and failure reasons.
3D visualization package	Element geometry, floor grouping, route edges, issue list, route graph binary, GLB model, app data JSON, selected door routes, route colors, stair blockers, and browser package paths.
Tests and audits	Unit test count, real PDF run, real IFC run, generated output files, extraction audit, candidate audit, governance audit, final correctness audit, generated SHACL parse check, SHACL report parse check, provider-name scan, and stale output cleanup.

6 Component Difficulty Review

6.1 PDF Ingestion

Main files: backend/document_intelligence/ingestion.py, backend/document_intelligence/pdf_parser.py, backend/document_intelligence/models.py

What was hard: A PDF is not like a spreadsheet. It does not tell the computer: “this sentence is a rule” or “this number belongs to this table row.” The same rule can appear as normal text, a table cell, a list item, or an image label.

What I tried: The PDF is split into separate objects: text chunks, table chunks, image chunks, and page metadata. This makes later steps easier because they know where evidence came from.

What works now: The real PDF was read and produced 1111 text chunks, 53 table chunks, and 210 image evidence chunks.

What is still weak: The parser does not fully understand legal section structure, exceptions, or references like “see section 4.3.7.” It extracts evidence, but it does not fully understand the document like a human expert.

6.2 Table Extraction

Main file: backend/document_intelligence/pdf_parser.py

What was hard: Important values are often inside tables. Door width was one example. The value existed, but earlier runs missed it because table meaning was not strong enough in the model input.

What I tried: Tables are stored as their own chunk type. Retrieval and candidate selection give table evidence its own weight.

What works now: The final door-width rule came from p012-table-001, which is direct table evidence.

What is still weak: Tables are still mostly stored as plain text. The system does not yet fully keep cell positions, headers, and row meaning.

6.3 Image Extraction And Ranking

Main files: `backend/document_intelligence/image_extraction.py`, `backend/document_intelligence/image_ranking.py`

What was hard: At first only embedded PDF images were processed by vision. That was dangerous because a PDF can draw diagrams, table borders, arrows, and measurement labels directly on the page. Those items are visible to a human, but they are not stored as normal embedded images.

What I tried: Now the system creates image evidence in three ways: embedded PDF images, rendered full-page images, and rendered table-region images. Rendering means the system makes a screenshot-like image of the PDF page or a table area. Ranking means the system gives each image a usefulness score before deciding which images go to the vision model.

Ranking parameters:

- **measurement_density:** how many measurements appear nearby.
- **numeric_density:** how many numbers appear.
- **normative_language:** words like minimum, maximum, must, shall, or German equivalents.
- **ocr_text_quality:** whether OCR found useful text.
- **page_context_density:** whether the surrounding page has many measurements or rule words.
- **table_context:** whether the page has tables.
- **visual_salience:** whether the image is visually large enough to matter.
- **uniqueness:** whether the image is not just a repeated logo or symbol.

What works now: All 210 image evidence chunks were ranked. The top 30 were sent to Qwen3-VL.

What is still weak: Only 30 out of 210 image evidence chunks were sent to the vision model. That is controlled by the vision budget. It is efficient, but it is still not full image coverage.

6.4 OCR

Main files: `backend/document_intelligence/providers.py`, `backend/document_intelligence/image_extraction.py`

What was hard: OCR can misread small text, rotated labels, German characters, and diagram labels.

What I tried: The active OCR provider is PaddleOCR. PaddleOCR is the component that reads text from extracted PDF images. The local runtime is pinned to `paddlepaddle==3.2.2` because `paddlepaddle==3.3.1` crashed during CPU inference on this laptop.

What works now: PaddleOCR executed on a real extracted image from the current PDF output and returned observable text. That proves the OCR provider is not only an interface; it runs on real image input.

The OCR model is now reused inside one Python process. Before that change, repeated ingestion loaded PaddleOCR again and again. A warm-up memory check showed repeated ingestion dropping from about 118 MB of extra traced Python memory to about 19 KB after reuse.

What is still weak: PaddleOCR is stronger than the old lightweight OCR path, but it is still OCR, not full document reasoning. It reads visible text. It does not understand the legal meaning of a diagram by itself; that job still belongs to retrieval, vision, and the requirement extraction agent.

6.5 Vision Processing

Main files: backend/document_intelligence/providers.py, backend/document_intelligence/image_ranking.py

What was hard: Vision models are slow on local hardware. Sending every image can take a long time. Sending too few images can miss rules.

What I tried: The system uses qwen3-vl:8b. The prompt asks for visible measurements, symbols, tables, diagrams, labels, limits, ranges, and rule wording. It no longer says wheelchair, door, ramp, width, or slope.

What works now: The real run sent 30 image evidence chunks to the vision model and produced non-empty vision text for 28 chunks.

What is still weak: The system proves that vision ran. It does not prove that every diagram was understood correctly. A benchmark with expected diagram rules is still needed.

6.6 Retrieval And Evidence Assembly

Main files: backend/requirement_extraction.py, backend/document_intelligence/image_ranking.py

What was hard: The missing-rule problem was not always caused by missing PDF text. The evidence was present, but the wrong evidence sometimes reached the model, or too much mixed evidence confused the model.

What I tried: Each evidence chunk gets a score. The model receives the strongest chunks first. Every selected and rejected chunk is recorded.

Scoring parameters:

- measurement_density
- constraint_likelihood
- normative_language
- definition_likelihood
- table_importance
- cross_reference
- citation
- requirement_confidence
- evidence_uniqueness
- semantic_centrality

What works now: The final run selected enough evidence to extract 4 final canonical rules. This is an improvement in evidence coverage, but it is not proof that every expected legal requirement was captured.

What is still weak: The scores are hand-made rules. They are explainable, but not proven by a large benchmark.

6.7 Requirement Extraction Agent

Main file: backend/requirement_extraction.py

What was hard: The agent is not allowed to invent missing rules. If it misses a rule, the system must fail instead of making one up.

What I tried: The model receives evidence and must return JSON canonical rules. It may not return SHACL, RDF, or ontology mappings. Bad outputs are rejected.

Important fix: With four evidence chunks per call, Qwen3.5 missed door width. Reducing the batch size to two evidence chunks fixed this for the latest run. This is not a DIN-specific trick. It simply gives the model less mixed context at once.

What works now: The real run made 48 model calls, accepted 16 outputs, rejected 2 outputs, and selected 5 final rules.

What is still weak: The model can still miss rules on a different PDF. The current success is not proof that every rule book will work.

6.8 Canonical Rule Model

Main files: `backend/canonical_rules.py`, `rules/canonical_rule_schema.json`

What was hard: The rule model must be neutral. It cannot already be SHACL. But it still needs enough data to generate SHACL later.

What I tried: Rules store rule ID, entity, property, operator, value, unit, source text, source page, confidence, and metadata. The supported rule list is in `rules/canonical_rule_schema.json`.

Schema parameters: Rule ID, description, entity, property, operator, value, unit, tolerance, and whether unit conversion should treat the value as a length.

What works now: The final rules were `corridor_width`, `door_width`, `ramp_slope`, `ramp_width`, and `turning_space`.

What is still weak: The rule model does not yet handle exceptions, conditions, scope, legal notes, or complex references.

6.9 Candidate Governance

Main files: `backend/requirement_extraction.py`, `backend/governance.py`

What was hard: The model can give high confidence to a weak candidate. So confidence alone is not safe.

What I tried: Candidates are ranked by evidence quality, not just model confidence.

Governance scores:

- semantic relevance
- measurement specificity
- source specificity
- requirement completeness
- ontology consistency
- retrieval support
- evidence density
- contradiction penalty

What works now: The latest run had 16 candidates, 5 selected candidates, 5 duplicate groups, and no candidate-level conflict records.

What is still weak: The weights are manually chosen. A proper dataset is needed to prove they are good.

6.10 Human Review

Main file: `backend/review_workflow.py`

What was hard: The system needs human approval, but there is no review UI yet.

What I tried: The command line can create explicit review records. Each decision stores reviewer, timestamp, rationale, candidate ID, and evidence references.

What works now: The latest run had 16 review queue records, 12 approved review records, 2 rejected review records, and 2 escalated review records.

What is still weak: A real UI is still needed. CLI flags are enough for testing, but not comfortable for real review work.

6.11 Rule Repository

Main file: `backend/rule_repository.py`

What was hard: Rules must be saved with proof of where they came from.

What I tried: Rules are saved as JSON with source document, chunk counts, metadata, and audit file references.

What works now: `output/app_package/canonical_rules.json` stores the final rules and provenance.

What is still weak: JSON is fine for a prototype. A real system would need a database, versioning, and access control.

6.12 Ontology Mapping

Main file: backend/ontology_mapping.py

What was hard: The system must connect a rule like `door_width` to the RDF property where door width is stored. The LLM should not guess this.

What I tried: The mapping layer is deterministic and reviewable.

Mapping parameters: Canonical rule ID, entity, property, ontology class, ontology property, mapping candidate ID, mapping score, mapping status, and mapping review decision.

What works now: The run created `ontology_mappings.json` and `ifc_mapping_audit.json`. Published rules had approved mapping reviews.

What is still weak: Mappings are still scoped to the current accessibility vocabulary. A different RDF graph would need new mappings.

6.13 SHACL Generation

Main file: backend/generated_shacl.py

What was hard: The LLM is not allowed to write SHACL. That means the Python generator must know exactly how to turn approved rules into shapes.

What I tried: SHACL is generated by deterministic Python templates. The old manual SHACL file was removed.

Generation parameters: Approved rule, approved mapping, target class, target property, operator, value, unit, datatype, NodeShape, PropertyShape, source page, evidence chunk ID, candidate ID, and review metadata.

What works now: The generated SHACL had 99 RDF triples and parsed successfully.

What is still weak: The generator supports only the current rule types. It does not support every kind of legal or engineering rule.

6.14 Publication Gate

Main file: backend/approved_rule_library.py

What was hard: A rule should not become SHACL just because it exists. It must have evidence, review, mapping, and no unresolved conflicts.

What I tried: Publication checks are run before the approved rule library is written.

Gate checks: Approved review exists, review queue exists, decision records exist, no unresolved conflicts, no unresolved duplicate groups, evidence exists, source chunk exists, confidence threshold passed, provenance complete, review identity complete, rule review approved, mapping review approved, and generated SHACL valid.

What works now: The latest publication status was **published**, with no failed publication checks.

What is still weak: The current review path is still command-line based.

6.15 pySHACL Validation

Main files: backend/shacl_runner.py, backend/approved_rule_library.py

What was hard: The SHACL rules and the RDF data graph must match. If the graph does not contain the needed property, SHACL cannot check it properly.

What I tried: Generated SHACL is parsed and then pySHACL runs it against the enriched RDF graph.

Validation parameters: Data graph path, SHACL file path, report path, target nodes, constraint results, conforms flag, and extracted issue records.

What works now: The final run reported 0 issues and the SHACL report parsed successfully.

What is still weak: Zero issues does not mean the whole building is legally compliant. It only means no violation was found for the generated rules and available graph data.

6.16 IFC Parsing And Geometry

Main files: backend/geometry.py, backend/ifc_tools.py

What was hard: IFC files are standardized, but every exported model can still store details differently. Door widths, space boundaries, and ramp data are not always easy to calculate.

What I tried: Python extracts building elements and adds derived measurements to the RDF graph.

Geometry parameters: IFC element type, GlobalId, name, floor, bounding box, center point, dimensions, declared width, derived door width, clear space width, turning space, ramp length, ramp usable width, ramp slope, and missing geometry.

What works now: The AC20 run extracted 312 elements, produced 317 route edges, and had 0 missing geometry records.

What is still weak: Another IFC file may store data differently. The current result does not prove full cross-IFC reliability.

6.17 IFCToLBD RDF Workflow

Main file: backend/ifc_tools.py

What was hard: The Java IFCToLBD tool prints ontology warnings and null-pointer messages in the current environment.

What I tried: The pipeline continues if the raw RDF graph is created, then Python enriches that graph with geometry and route data.

What works now: The output package was written successfully.

What is still weak: The Java warnings should be cleaned up for a serious deployment.

6.18 Route Graph And 3D View

Main files: backend/routes.py, backend/package_writer.py, backend/glb_export.py, frontend/app.js

What was hard: The visual route view must match the data used by SHACL. Otherwise the browser may show one thing while validation checks another.

What I tried: The browser uses the data package created by the preprocessing run. It does not calculate routes again.

Route parameters: Door-to-space links, route edge ID, start door, end door, route path, route width, door width, turning space, ramp intersection, stair intersection, route status, and failure reasons.

What works now: The latest run created 317 route edges and wrote the browser package.

What is still weak: The browser is a viewer. It is not an independent validator.

6.19 Testing

Main folder: tests/

What was hard: It was not enough to create interfaces. Tests had to show that the components actually run.

What I tried: Tests cover text PDF, scanned PDF OCR, image vision, rule extraction, governance gates, SHACL generation, and pySHACL validation.

What works now: The latest test command ran 19 tests and passed:

```
python -m unittest discover -s tests -p "test_*.py" -v
```

A repeated-ingestion memory check also passed. After warming up PaddleOCR once, three repeated ingestion runs created the expected image chunks and OCR text with only about 19 KB traced Python memory growth.

What is still weak: The tests are not a full benchmark. They do not prove performance on many different rule books and IFC files.

7 Hardest Problems

7.1 Evidence Was Present But Not Used Well

The system had the needed text, but the model did not always use it. This showed that extraction is not only about reading the PDF. It is also about sending the right evidence to the model in the right amount.

7.2 Avoiding Fake Fixes

It would have been easy to hardcode known values like 90 cm or 150 cm. That would make the demo look better, but it would be wrong. The system now avoids creating missing rules by repair code.

7.3 Local Model Behavior

Qwen3.5 needed JSON mode with thinking disabled. Without that, the response could be empty even when the model internally produced JSON.

7.4 Image Budget

The old limit of 5 images was too small. Ranking all images and sending 30 to vision is better, but full vision coverage is still not done.

8 Current Classification

Component	Current state	Reason
PDF ingestion	Research ready	It works on the real PDF, but does not fully understand legal document structure.
OCR	Research ready	It runs and produces text, but is not benchmarked against stronger OCR models.
Vision	Research ready	It runs on selected image evidence, but only 30 out of 210 chunks were sent to vision and 28 produced non-empty vision output.
Retrieval	Research ready, improving	It is scored and audited, but the scoring is hand-made.
Requirement extraction	Research ready	It extracted 4 final canonical rules in the latest real run, but model behavior is prompt-sensitive and still needs a benchmark.
Canonical rule model	Research ready	It is useful, but does not handle exceptions or legal scope yet.
Governance	Early pilot direction	It has provenance and review records, but no real review UI.
Ontology mapping	Research ready	It is deterministic, but tied to the current vocabulary.
SHACL generation	Pilot direction for current rules	It is deterministic, but only supports current templates.
pySHACL validation	Pilot ready for current graph	It runs correctly, but depends on complete RDF graph data.

IFC/RDF workflow	Research ready	It works for AC20. IFCtoLBD Java output is now captured in a log file, but the converter still reports internal ontology warnings there.
3D view	Demo ready	It shows prepared data, but it is not a validator.

9 Final Conclusion

The system is now much better than a manual SHACL-only prototype. It can read a real PDF, extract text, tables, and images, run OCR, run a vision model on ranked images, extract canonical rules with a local model, review them, map them to RDF properties, generate SHACL, run pySHACL, and prepare the browser output.

The most difficult parts were not basic coding tasks. The hardest parts were choosing the right evidence, keeping the model from inventing rules, making every rule traceable, and making sure the system fails honestly when the evidence or model output is not good enough.

The system works for the tested DIN wheelchair-route flow and the AC20 IFC file. It should not yet be described as a general production rule-book-to-SHACL generator. The biggest missing pieces are stronger document parsing, stronger OCR or document vision, full image coverage or a better image budget strategy, a real review UI, a larger test benchmark, broader ontology mappings, and cleaner IFCtoLBD setup.